

OptiCrop: Smart Agricultural Production Optimization Engine

Project Description:

The OptiCrop: Smart Agricultural Production Optimization Engine is an Artificial Intelligence and Machine Learning-based web application designed to assist farmers in selecting the most suitable crop for cultivation based on soil nutrients and environmental conditions. Agriculture plays a vital role in ensuring food security and supporting the economy, but traditional crop selection methods often depend on personal experience, local practices, and unpredictable weather conditions. These methods may lead to poor crop selection, lower yields, and inefficient use of agricultural resources.

The OptiCrop system addresses these challenges by using a trained Machine Learning model that analyzes multiple agricultural parameters, including Nitrogen (N), Phosphorus (P), Potassium (K), temperature, humidity, soil pH, and rainfall. Based on these inputs, the model predicts the crop that is most suitable for cultivation under the given conditions.

The application is developed using the Flask web framework, which integrates the frontend interface with the machine learning prediction model. The frontend is built using HTML, CSS, and JavaScript to provide a simple, responsive, and user-friendly interface. Users can enter soil and environmental data through the web application and instantly receive crop recommendations.

The machine learning model is trained using the Crop Recommendation Dataset obtained from Kaggle. The dataset contains agricultural information collected from different crops and environmental conditions. During model development, the data was preprocessed, analyzed, and used to train various machine learning algorithms. The best-performing model was selected based on prediction accuracy and saved for deployment.

The project demonstrates the practical application of Artificial Intelligence in agriculture by enabling data-driven farming decisions. The solution aims to improve crop productivity, reduce resource wastage, and help farmers make informed decisions that increase agricultural efficiency and sustainability.

Scenarios

Scenario 1– Farmer Crop Selection:

A farmer wants to identify the most suitable crop based on soil nutrients and weather conditions before cultivation. The OptiCrop system analyzes the provided data and recommends the best crop for maximum productivity.

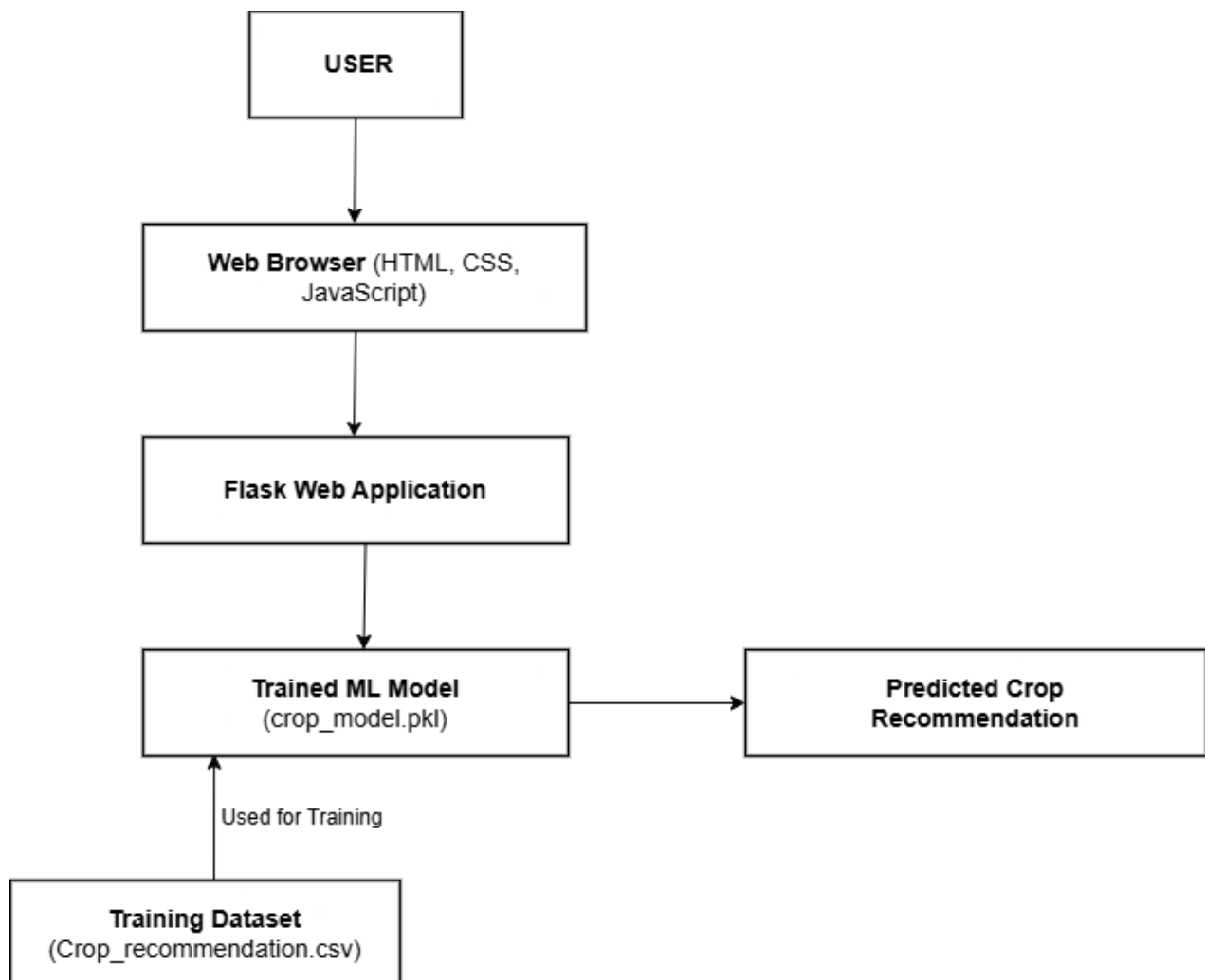
Scenario 2– Agricultural Consultant:

An agricultural consultant receives soil test results from a farmer. Using the OptiCrop application, the consultant enters the soil parameters and obtains an AI-based crop recommendation to guide the farmer.

Scenario 3– Agriculture Student / Research:

An agriculture student uses the OptiCrop application to study the relationship between soil nutrients, environmental conditions, and crop suitability. The application helps demonstrate how machine learning can support precision agriculture.

Technical Architecture:



Description:

The OptiCrop system follows a layered architecture consisting of a presentation layer, an application layer, and a machine learning layer. Users access the application through a web browser, where they enter soil nutrient and environmental details. The Flask web application receives and validates the input before passing it to the trained machine learning model (crop_model.pkl). The model, which was trained using the Crop Recommendation dataset (CSV file), predicts the most suitable crop based on the provided parameters. The prediction is then returned to the Flask application and displayed to the user through the web interface.

Note:

The OptiCrop project does not use a relational database. The application uses a CSV dataset for training and a serialized machine learning model (crop_model.pkl) for prediction. Therefore, an Entity Relationship (ER) Diagram is not applicable.

Pre-requisites:

Before running the OptiCrop application, ensure the following software, tools, and technologies are available on your system.

Python Programming

Python is the primary programming language used for implementing the machine learning model, data preprocessing, backend logic, and Flask application.

Topics Required:

- Variables
- Functions
- Lists
- Dictionaries
- File Handling
- Exception Handling
- Object-Oriented Programming (Basic)

Documentation:

<https://docs.python.org/3/>

Flask Framework

Flask is a lightweight Python web framework used to build the backend application and connect the machine learning model with the frontend user interface.

Knowledge Required:

- Flask Routing
- Templates
- Request Handling
- Form Processing
- Static Files
- Jinja2 Templates

Documentation:

<https://flask.palletsprojects.com/>

Machine Learning Libraries

The project uses several Python libraries for machine learning, data preprocessing, and visualization.

Required Libraries:

- Pandas
- NumPy
- Scikit-learn
- Joblib
- Matplotlib
- Seaborn

Documentation:

<https://scikit-learn.org/>

<https://pandas.pydata.org/>

<https://numpy.org/>

HTML, CSS and JavaScript

The frontend interface is designed using HTML, CSS, and JavaScript.

Knowledge Required:

- HTML Forms
- CSS Styling
- Responsive Design
- JavaScript Basics

Documentation:

<https://developer.mozilla.org/>

or

<https://www.w3schools.com/>

Git and GitHub

Git is used for version control, while GitHub is used to host the project repository and manage collaboration among team members.

Knowledge Required:

- Repository Creation
- Git Clone

- Git Commit
- Git Push
- Pull Requests

Documentation:

<https://git-scm.com/doc>

Visual Studio Code

Visual Studio Code is used as the primary development environment for editing Python, HTML, CSS, JavaScript, and Flask files.

Documentation:

<https://code.visualstudio.com/docs>

Development Environment

The project should be executed inside a Python Virtual Environment.

Create Virtual Environment:

```
python -m venv venv
```

Activate Environment (Windows):

```
venv\Scripts\activate
```

Install Dependencies:

```
pip install -r requirements.txt
```

Run the Flask Application:

```
python app.py
```

Open the Application:

```
http://127.0.0.1:5000
```

Project Workflow:

Milestone 1: Model Selection and Architecture

- **Activity 1.1: Collect the Crop Recommendation Dataset**

Collect the Crop Recommendation dataset from Kaggle and store it in the project directory for preprocessing and model training.

- **Activity 1.2: Research and Select the Appropriate Machine Learning Model**
Analyze different Machine Learning algorithms such as Logistic Regression, Decision Tree, Random Forest, K-Nearest Neighbors, and Support Vector Machine. Compare their performance and select the best model for crop prediction.
- **Activity 1.3: Define the Architecture of the Application**
Design the overall architecture of the OptiCrop application by defining the interaction between the user interface, Flask backend, Machine Learning model, dataset, and prediction module.
- **Activity 1.4: Set Up the Development Environment**
Install Python, Visual Studio Code, Flask, required Python libraries, and configure the project environment for development.

Milestone 2: Core Functionalities Development

- **Activity 2.1: Develop the Crop Recommendation Model**
Perform data preprocessing, exploratory data analysis, feature selection, model training, testing, and save the trained Machine Learning model.
- **Activity 2.2: Implement the Flask Backend**
Develop the Flask backend to process user inputs, load the trained model, generate crop predictions, and return the prediction results to the frontend.

Milestone 3: App.py Development

- **Activity 3.1: Develop the Main Application Logic**
Create the app.py file to define application routes, integrate the Machine Learning model, receive user input, process predictions, and display the recommended crop.

Milestone 4: Frontend Development

- **Activity 4.1: Design the User Interface**
Develop responsive web pages using HTML and CSS to allow users to enter agricultural parameters and interact with the application.
- **Activity 4.2: Implement Frontend Functionality**
Use JavaScript to improve user interaction, validate input fields, and display prediction results dynamically.

Milestone 5: Deployment

- **Activity 5.1: Test the Application Locally**
Run the Flask application on the local machine and verify all functionalities before deployment.
- **Activity 5.2: Deploy the Application Using Ngrok**
Expose the locally running Flask application to the internet using Ngrok and generate a public URL for demonstration.
- **Activity 5.3: Validate the Deployed Application**
Test all application features using the public URL and verify that the deployed application works correctly.

Milestone 6: Project Completion

- **Activity 6.1: Final Testing and Documentation**
Perform complete system testing, verify prediction accuracy, capture screenshots, and prepare the final project documentation.

Milestone 1: Model Selection and Architecture

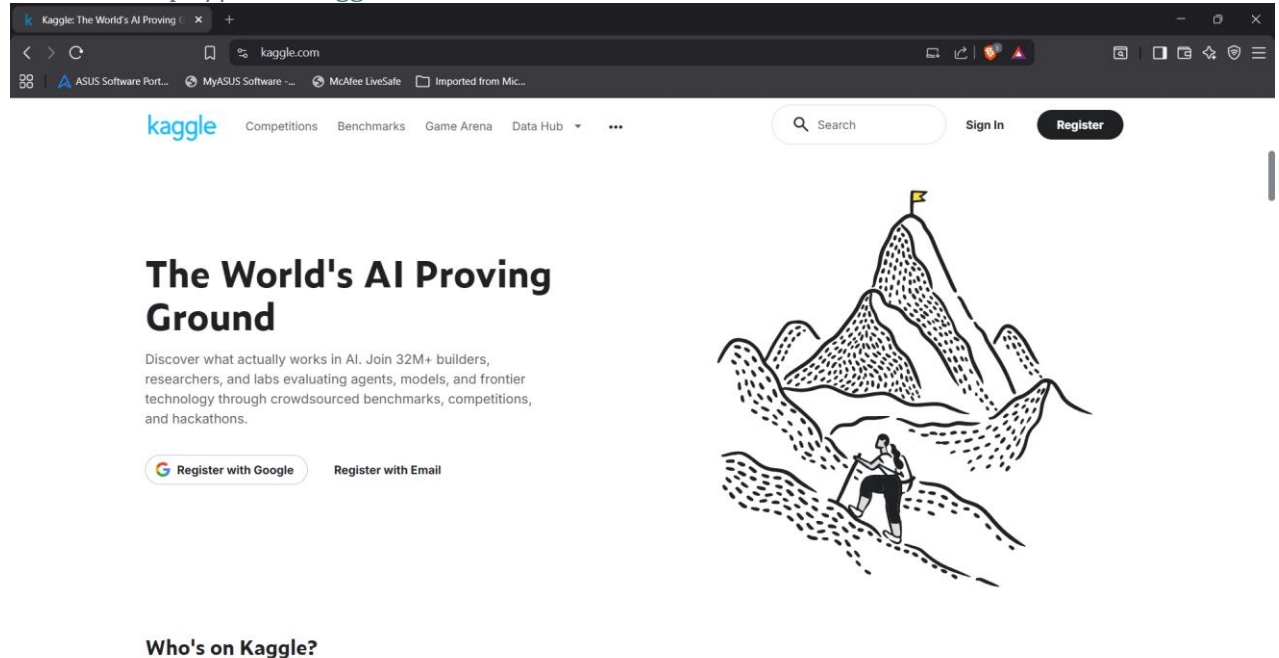
In this milestone, we focus on selecting the appropriate Machine Learning approach for the OptiCrop project and designing the overall architecture of the application. This involves collecting a suitable dataset, researching different Machine Learning algorithms, defining the interaction between the frontend, backend, and prediction model, and setting up the complete development environment. These activities establish the foundation for building an intelligent crop recommendation system that provides accurate predictions based on soil nutrients and environmental conditions.

Activity 1.1: Collect the Crop Recommendation Dataset

Before developing the Machine Learning model, it is essential to collect a reliable and well-structured dataset. The Crop Recommendation Dataset contains agricultural parameters such as Nitrogen (N), Phosphorus (P), Potassium (K), temperature, humidity, pH, and rainfall, along with the corresponding crop labels. This dataset is used for preprocessing, training, testing, and evaluating the Machine Learning models.

- **Step 1 — Open the Kaggle Website:** Open your web browser and visit the Kaggle website.

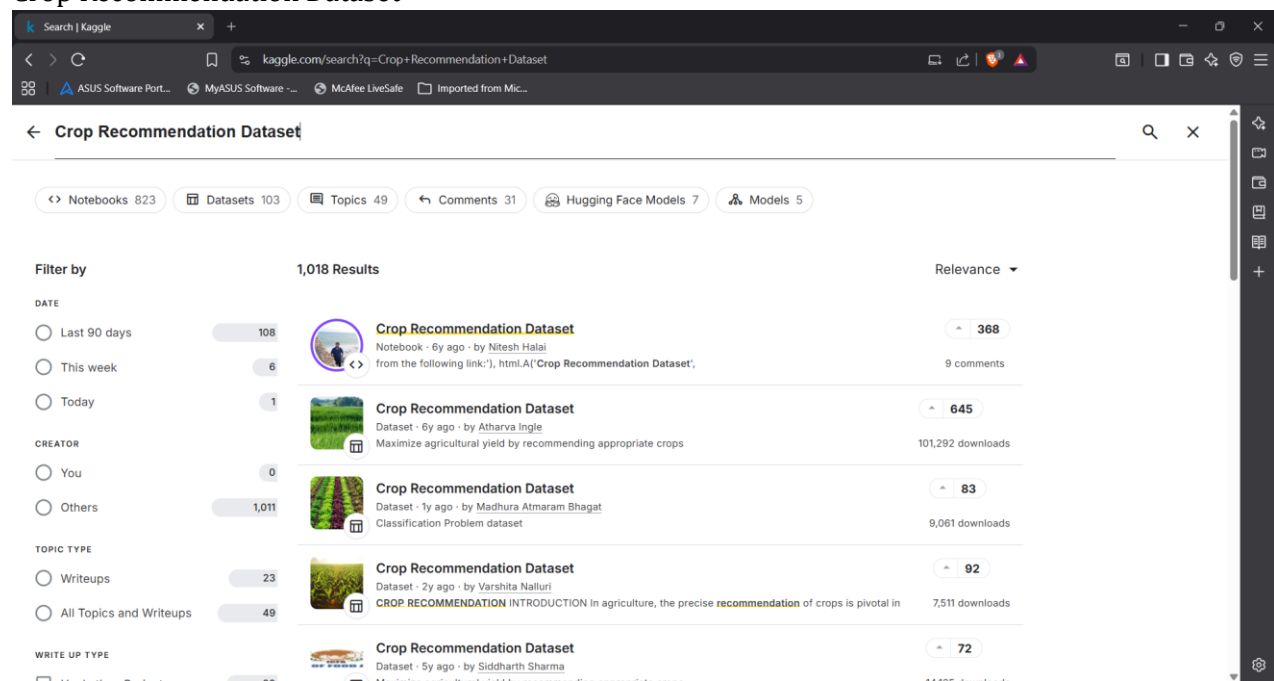
Website: <https://www.kaggle.com>



- **Step 2 — Search for the Dataset:** Search for **Crop Recommendation Dataset** in the Kaggle search bar.

Dataset Name:

Crop Recommendation Dataset

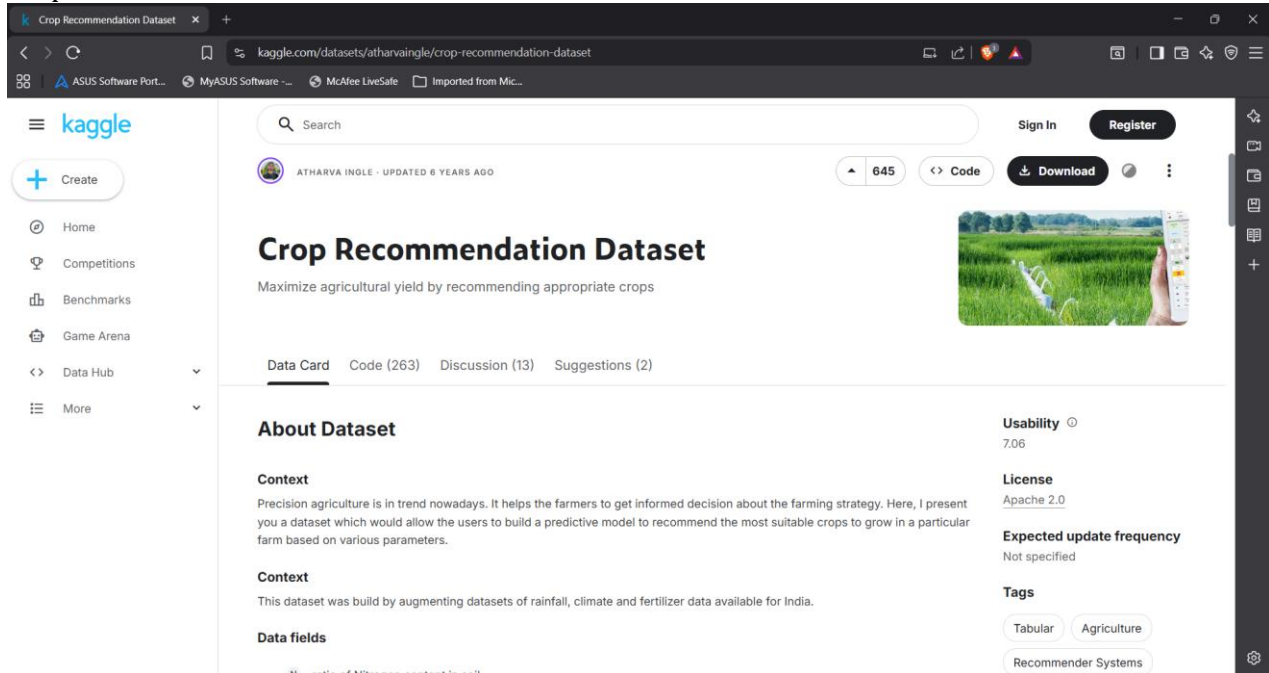


- **Step 3 — Download the Dataset**

Open the dataset page and click the Download button. Extract the downloaded ZIP file.

The extracted file is:

Crop_recommendation.csv

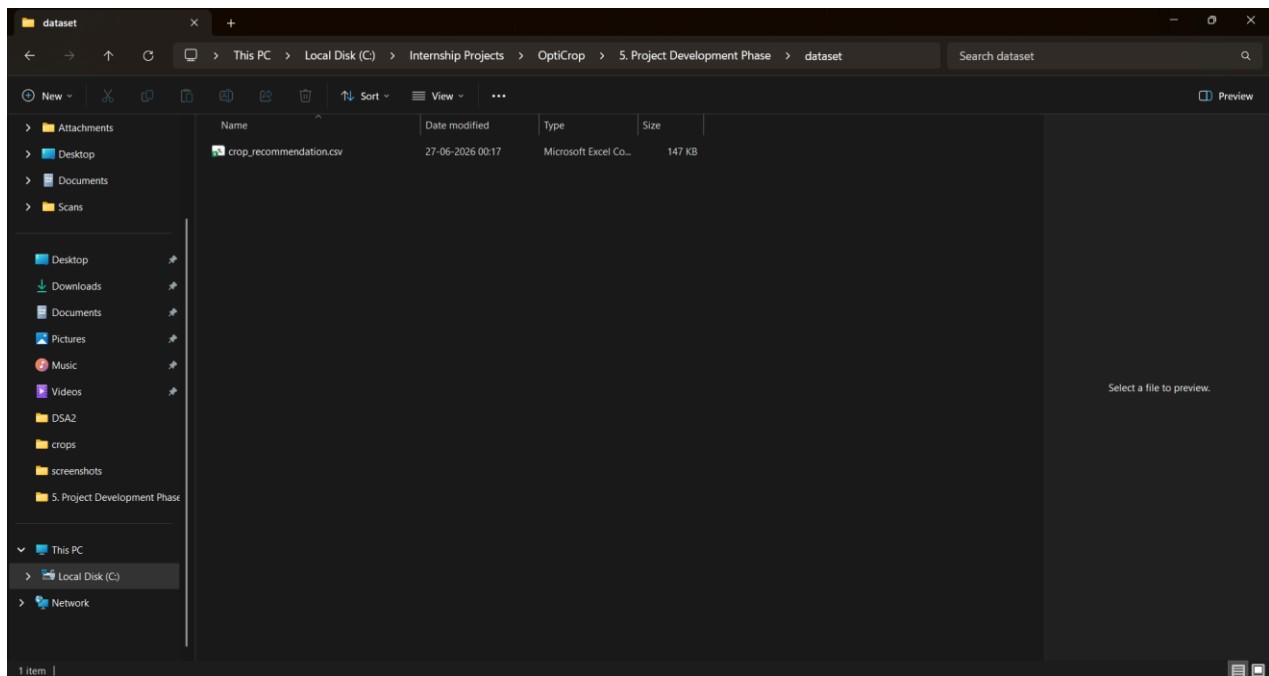


- **Step 4 — Store the Dataset:** Create a folder named dataset inside the project directory and place the downloaded CSV file inside it.

OptiCrop/

```

|
├── dataset/
│   └── Crop_recommendation.csv
|
├── model/
├── static/
├── templates/
├── app.py
└── requirements.txt
  
```



Dataset Description

The dataset consists of agricultural attributes that are used to predict the most suitable crop for cultivation.

Feature	Description
N	Nitrogen content in soil
P	Phosphorus content in soil
K	Potassium content in soil
Temperature	Temperature in degree Celsius
Humidity	Relative humidity (%)
pH	Soil pH value
Rainfall	Rainfall (mm)
Label	Recommended crop

Activity 1.2: Research and Select the Appropriate Machine Learning Model

Selecting an appropriate Machine Learning algorithm is one of the most important stages in developing the OptiCrop application. Different classification algorithms are researched and evaluated to identify the model that provides the highest prediction accuracy for the Crop Recommendation Dataset. The evaluation is based on factors such as prediction accuracy, reliability, execution time, and suitability for agricultural data. After comparing the performance of multiple algorithms, the best-performing model is selected for crop prediction and prepared for integration with the Flask web application.

The Machine Learning algorithms considered for evaluation include:

- Logistic Regression
- Decision Tree Classifier
- Random Forest Classifier
- K-Nearest Neighbors (KNN)
- Support Vector Machine (SVM)

The selected model is trained using the processed dataset and saved as `crop_model.pkl`, which is later used by the Flask backend to generate crop recommendations based on user-provided agricultural parameters.

Model Comparison	
Algorithm	Accuracy
Decision Tree	0.9863636363636363
K-Nearest Neighbors (KNN)	0.9704545454545455
Logistic Regression	0.9522727272727273
Random Forest	0.9931818181818182

Activity 1.3: Define the Architecture of the Application

The OptiCrop application follows a three-layer architecture consisting of the Presentation Layer, Application Layer, and Machine Learning Layer. The architecture defines how users interact with the web application, how requests are processed by the Flask backend, and how the trained Machine Learning model generates crop recommendations. A well-defined architecture improves scalability, maintainability, and the overall performance of the application.

The following activities were performed while defining the application architecture:

- **Draft an Architecture Diagram:**
Create a technical architecture diagram illustrating the interaction between the user, web browser, Flask application, Machine Learning model, training dataset, and prediction results.

- **Define the Presentation Layer:** Design a responsive web interface using HTML, CSS, and JavaScript that allows users to enter soil nutrient values and environmental conditions for crop prediction.
- **Define the Application Layer:** Configure the Flask backend to receive user input, validate the entered values, load the trained Machine Learning model, and process prediction requests.
- **Define the Machine Learning Layer:** Integrate the trained Machine Learning model (crop_model.pkl) with the Flask application so that predictions can be generated efficiently based on user inputs.
- **Define Data Flow:** Establish the complete workflow where users provide agricultural parameters through the web interface, Flask processes the request, the Machine Learning model predicts the most suitable crop, and the recommendation is displayed on the results page.

Activity 1.4: Set Up the Development Environment

A proper development environment is required for building, training, testing, and deploying the OptiCrop application. Python, Flask, Machine Learning libraries, and development tools are installed before implementing the project.

The following steps were performed to configure the development environment:

- **Install Python:** Install Python (version 3.10 or later) along with the pip package manager.
- **Create a Virtual Environment:** Create and activate a virtual environment to isolate the project dependencies.

```
python -m venv .venv
```

Activate the virtual environment (Windows):

```
.venv\Scripts\activate
```

- **Install Required Libraries:** Install all required dependencies using the requirements.txt file.

```
pip install -r requirements.txt
```

The required libraries include:

- Flask
- Pandas
- NumPy
- Scikit-learn
- Matplotlib
- Seaborn
- Joblib
- **Configure the Project Structure:** Organize the project folders, including the dataset, trained model, templates, static resources, and application files.

Example project structure:

5. Project Development Phase

```
|
|— dataset
|   └─ Crop_recommendation.csv
|
|— model
|   └─ crop_model.pkl
|
|— notebooks
|   └─ data_preprocessing.ipynb
|
|— static
|   ├── css
|   ├── js
|   └─ images
|
|— templates
|   ├── index.html
|   └─ result.html
|
|— app.py
|— requirements.txt
|— README.md
└─ .gitignore
```

- **Verify the Installation:** Run the Flask application locally to ensure that all dependencies are installed correctly and that the development environment is configured successfully.

python app.py

Open the application in a web browser:

<http://127.0.0.1:5000>

Milestone 2: Core Functionalities Development

This milestone focuses on implementing the core functionalities of the OptiCrop application. The project involves preprocessing the agricultural dataset, training Machine Learning models, evaluating their performance, selecting the best model, and integrating the prediction system with the Flask application to recommend suitable crops based on soil and environmental conditions.

Activity 2.1: Data Preprocessing and Exploratory Data Analysis

The Crop Recommendation dataset was loaded into Jupyter Notebook for preprocessing and analysis. The dataset was examined to verify data types, identify missing values, and understand the statistical distribution of each feature. Since the dataset contained no missing values, no additional data cleaning was required.

Exploratory Data Analysis (EDA) was performed using various visualization techniques such as histograms, correlation heatmaps, and crop distribution charts. These visualizations helped understand the relationships between soil nutrients, climatic parameters, and crop labels before training the Machine Learning models.

Activity 2.2: Machine Learning Model Development

Different Machine Learning classification algorithms were trained using the processed dataset to identify the most accurate crop prediction model. The dataset was divided into training and testing datasets using the Train-Test Split technique.

The following Machine Learning algorithms were evaluated:

- Logistic Regression
- Decision Tree Classifier
- Random Forest Classifier
- K-Nearest Neighbors (KNN)
- Support Vector Machine (SVM)

Each algorithm was trained using the agricultural dataset, and its performance was evaluated using prediction accuracy. The model with the highest accuracy was selected for deployment in the OptiCrop application.

Activity 2.3: Model Evaluation and Saving the Best Model

After training multiple Machine Learning models, their performances were compared using the Accuracy Score metric. The algorithm producing the highest prediction accuracy was selected as the final model.

The trained model was then serialized and stored using the Pickle library as `crop_model.pkl`. Saving the trained model allows the Flask application to perform predictions efficiently without retraining the model whenever the application starts.

Activity 2.4: Crop Prediction Implementation

The saved Machine Learning model was integrated with the Flask web application. Users enter agricultural parameters including Nitrogen, Phosphorus, Potassium, Temperature, Humidity, pH, and Rainfall through the web interface.

The application processes these inputs and passes them to the trained Machine Learning model. Based on the learned patterns from the training dataset, the model predicts the most suitable crop for cultivation. The predicted crop recommendation is then displayed on the user interface, enabling farmers to make informed agricultural decisions.

Milestone 3: App.py Development

This milestone focuses on developing the Flask backend of the OptiCrop application. The application loads the trained Machine Learning model, accepts agricultural parameters from the user, predicts the most suitable crop, and displays the recommendation through a web interface.

Activity 3.1: Writing the Main Application Logic in app.py

Create the main Flask application file (app.py) to manage the entire workflow of the OptiCrop system.

Import the required Python libraries including Flask, NumPy, Pickle, and the crop information module. Initialize the Flask application and load the trained Machine Learning model (crop_model.pkl) using the Pickle library.

The application defines routes for the Home page, About page, and Prediction page. The POST route accepts user input, processes the agricultural parameters, predicts the most suitable crop using the trained Machine Learning model, and returns the prediction result to the web page.

```
from flask import Flask, render_template, request
import pickle
import numpy as np
import os
from crop_data import crop_info

app = Flask(__name__)

# -----
# Load Trained Model
# -----
with open("model/crop_model.pkl", "rb") as file:
    model = pickle.load(file)

# -----
# Home Page
# -----
@app.route("/")
def home():
    return render_template("home.html")

# -----
# About Page
# -----
@app.route("/about")
def about():
    return render_template("about.html")

# -----
# Prediction Page
# -----
@app.route("/predict", methods=["GET"])
def predict_page():
    return render_template("predict.html")

# -----
# Predict Crop
# -----
@app.route("/predict", methods=["POST"])
def predict():
```

Activity 3.2: Loading the Trained Machine Learning Model

The trained Machine Learning model is loaded when the Flask application starts.

The Pickle library is used to deserialize the saved model (crop_model.pkl), allowing the application to make predictions without retraining the model each time the application is executed.

This improves the application's efficiency and reduces prediction time.

```
with open("model/crop_model.pkl", "rb") as file:
    model = pickle.load(file)
```

Activity 3.3: Processing User Input

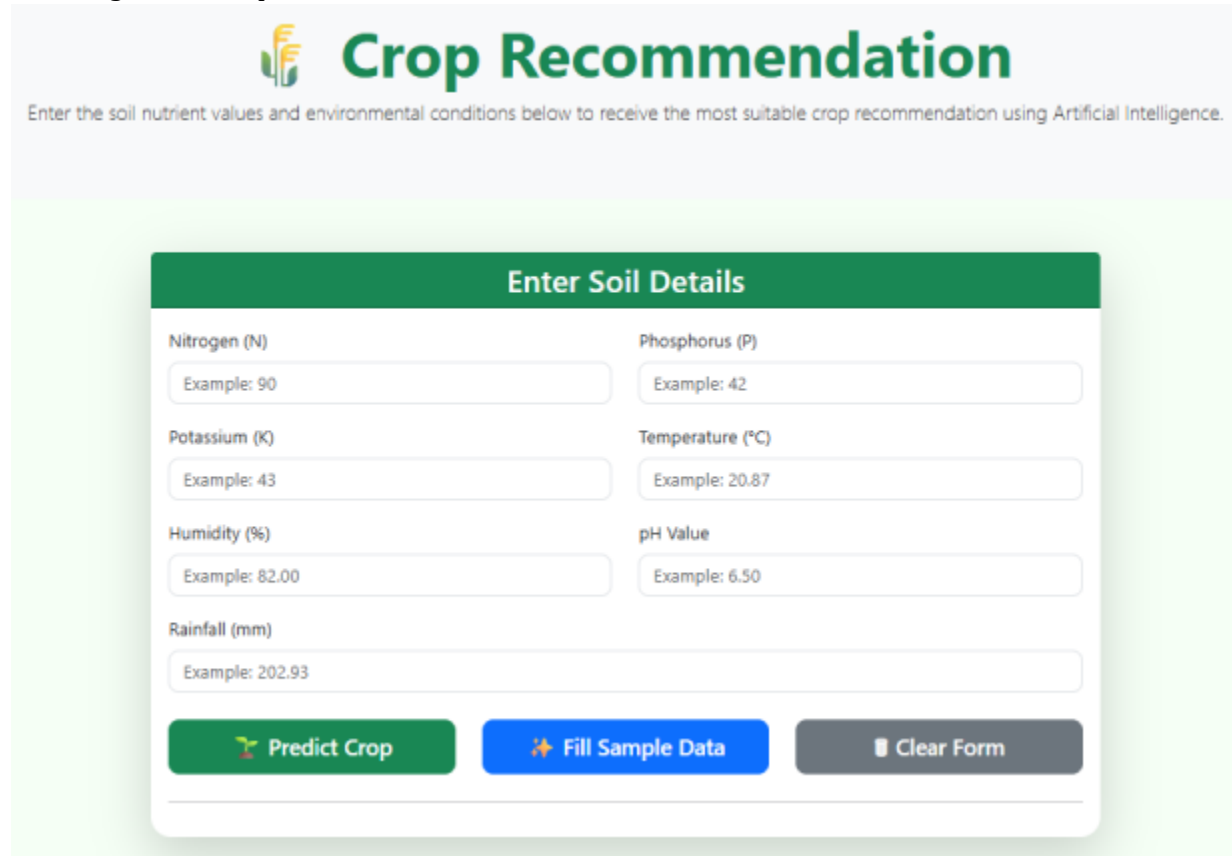
The Flask application receives agricultural parameters entered by the user through the HTML form.

The following parameters are collected:

- Nitrogen (N)
- Phosphorus (P)
- Potassium (K)
- Temperature
- Humidity
- pH

- Rainfall

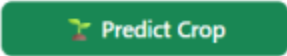
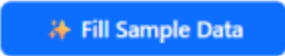
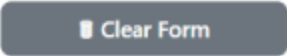
The received values are converted into numerical format before being passed to the Machine Learning model for prediction.



Crop Recommendation

Enter the soil nutrient values and environmental conditions below to receive the most suitable crop recommendation using Artificial Intelligence.

Enter Soil Details	
Nitrogen (N) Example: 90	Phosphorus (P) Example: 42
Potassium (K) Example: 43	Temperature (°C) Example: 20.87
Humidity (%) Example: 82.00	pH Value Example: 6.50
Rainfall (mm) Example: 202.93	

Activity 3.4: Generating and Displaying the Prediction Result

The processed input values are passed to the trained Machine Learning model to predict the most suitable crop.

The application also calculates the prediction confidence using probability scores returned by the model. The predicted crop, confidence percentage, and additional crop information are then displayed on the result page using Flask's `render_template()` function.

This enables farmers to receive an accurate crop recommendation instantly through the web application.

```
<hr class="my-5">

{% if prediction %}

<section id="result" class="py-5">

<div class="container">

<div class="card shadow-lg border-0">

<div class="card-header bg-success text-white text-center">

<h2> 🌾 Prediction Result</h2>

</div>

<div class="card-body">

<div class="row align-items-center">

<!-- Crop Image -->

<div class="col-lg-4 text-center">



<h2 class="mt-4 text-success fw-bold">

{{ prediction }}
```

Milestone 4: Frontend Development

This milestone focuses on designing and developing the user interface of the OptiCrop application. The frontend was built using HTML, CSS, and JavaScript to provide a simple, responsive, and user-friendly interface. It enables users to enter agricultural parameters, submit data for crop prediction, and view the recommended crop along with additional information.

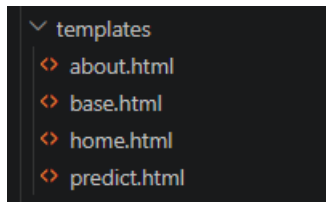
Activity 4.1: Designing the User Interface

Create the user interface of the OptiCrop application using HTML and CSS.

The application consists of three main pages:

- **Home Page** – Introduces the OptiCrop project and its objectives.
- **About Page** – Provides information about the project and the technologies used.
- **Prediction Page** – Allows users to enter agricultural parameters for crop recommendation.

The interface is designed with a responsive layout, navigation bar, attractive color scheme, and properly aligned input fields to provide a better user experience.



Activity 4.2: Creating the Prediction Form

Develop the crop prediction form using HTML.

The form collects the following agricultural parameters from the user:

- Nitrogen (N)
- Phosphorus (P)
- Potassium (K)
- Temperature
- Humidity
- pH
- Rainfall

Each input field includes placeholder values and validation to ensure correct user input before submitting the form.

```
<section id="prediction-form" class="py-5">

<div class="container">

<div class="row justify-content-center">

<div class="col-lg-8">

<div class="card shadow-lg border-0">

<div class="card-header bg-success text-white">

<h3 class="text-center mb-0">

Enter Soil Details

</h3>

</div>

<div class="card-body">

{% if error %}

<div class="alert alert-danger">

{{ error }}

</div>

{% endif %}

<form action="/predict" method="POST">
```

Activity 4.3: Styling the Web Application

Apply CSS styling to improve the appearance and usability of the application.

The following design elements were implemented:

- Responsive layout
- Navigation bar
- Cards and containers
- Styled input fields
- Color-themed buttons
- Footer section
- Mobile-friendly interface

These improvements provide a clean, modern, and user-friendly interface for farmers and agricultural consultants.

```
body{
  font-family: "Segoe UI", sans-serif;
  background: #f5fff5;
  color: #333;
}

/* Navbar */

.navbar{
  box-shadow: 0 3px 10px rgba(0,0,0,.15);
}

/* Hero */

.display-3{
  font-weight: 700;
}

.lead{
  font-size: 18px;
}

/* Cards */

.card{
  border: none;
  border-radius: 18px;
  box-shadow: 0 10px 25px rgba(0,0,0,.10);
  transition: .3s;
}

.card:hover{
  transform: translateY(-5px);
}

/* Buttons */

.btn{
  border-radius: 10px;
  font-weight: 600;
}

.btn-success{
  background: #198754;
}
```

Activity 4.4: Displaying the Prediction Result

After the user submits the agricultural parameters, the predicted crop and additional information are displayed on the result page.

The result page presents:

- Recommended Crop
- Prediction Confidence

- Crop Image
- Suitable Season
- Soil Type
- Water Requirement
- Fertilizer Recommendation
- Harvest Time
- Farming Tips

This information helps users make informed agricultural decisions based on the Machine Learning prediction.

```
const sampleData = [
  {
    nitrogen:90, phosphorus:42, potassium:43, temperature:20.87, humidity:82, ph:6.5, rainfall:202.93 // Rice
  },
  {
    nitrogen:77, phosphorus:58, potassium:30, temperature:25, humidity:60, ph:6.2, rainfall:85 // Maize
  },
  {
    nitrogen:40, phosphorus:67, potassium:79, temperature:20, humidity:18, ph:7.2, rainfall:80 // Chickpea
  },
  {
    nitrogen:100, phosphorus:75, potassium:50, temperature:27, humidity:80, ph:6.5, rainfall:105 // Banana
  },
  {
    nitrogen:20, phosphorus:27, potassium:30, temperature:30, humidity:50, ph:5.8, rainfall:95 // Mango
  },
  {
    nitrogen:24, phosphorus:132, potassium:198, temperature:24, humidity:81, ph:6.5, rainfall:70 // Grapes
  },
  {
    nitrogen:20, phosphorus:130, potassium:200, temperature:22, humidity:90, ph:6.2, rainfall:110 // Apple
  },
  {
    nitrogen:21, phosphorus:16, potassium:30, temperature:27, humidity:95, ph:5.5, rainfall:175 // Coconut
  },
  {
    nitrogen:120, phosphorus:40, potassium:20, temperature:25, humidity:80, ph:6.0, rainfall:95 // Cotton
  },
  {
    nitrogen:101, phosphorus:15, potassium:40, temperature:24, humidity:55, ph:6.0, rainfall:150 // Coffee
  }
];

function fillSampleData() {

  const sample = sampleData[Math.floor(Math.random() * sampleData.length)];

  document.getElementsByName("nitrogen")[0].value = sample.nitrogen;
  document.getElementsByName("phosphorus")[0].value = sample.phosphorus;
  document.getElementsByName("potassium")[0].value = sample.potassium;
  document.getElementsByName("temperature")[0].value = sample.temperature;
  document.getElementsByName("humidity")[0].value = sample.humidity;
  document.getElementsByName("ph")[0].value = sample.ph;
  document.getElementsByName("rainfall")[0].value = sample.rainfall;
}

window.onload = function () {

  const result = document.getElementById("result");

  if(result){
    result.scrollIntoView({
      behavior:"smooth"
    });
  }
};
```

Milestone 5: Project Testing

This milestone focuses on testing the OptiCrop application to ensure that all functionalities work correctly. The Machine Learning model, Flask backend, frontend interface, and prediction workflow were tested to verify accurate crop recommendations and a smooth user experience.

Activity 5.1: Testing the User Interface

The user interface was tested to ensure that all web pages load correctly and are responsive. Navigation links, buttons, input fields, and page layouts were verified across the application.

The following pages were tested:

- Home Page
- About Page
- Prediction Page

The interface was verified for proper alignment, readability, responsiveness, and ease of use.

Enter Soil Details

Nitrogen (N)	Phosphorus (P)
20	27
Potassium (K)	Temperature (°C)
30	30
Humidity (%)	pH Value
50	5.8
Rainfall (mm)	
95	

Predict Crop

Fill Sample Data

Clear Form

Activity 5.2: Testing the Crop Prediction Functionality

The crop recommendation functionality was tested by entering different agricultural parameters such as Nitrogen, Phosphorus, Potassium, Temperature, Humidity, pH, and Rainfall.

The application successfully processed the input values and generated the most suitable crop recommendation along with prediction confidence and additional crop information.

This verified that the Machine Learning model was correctly integrated with the Flask application.

```
PS C:\Internship Projects\OptiCrop> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& "c:\Internship Projects\OptiCrop\.venv\Scripts\Activate.ps1")
(.venv) PS C:\Internship Projects\OptiCrop> cd "5. Project Development Phase"
(.venv) PS C:\Internship Projects\OptiCrop\5. Project Development Phase> python app.py
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://192.168.0.100:5000
Press CTRL+C to quit
192.168.0.100 - - [07/Jul/2026 12:10:26] "GET / HTTP/1.1" 200 -
192.168.0.100 - - [07/Jul/2026 12:10:26] "GET /static/css/style.css HTTP/1.1" 304 -
192.168.0.100 - - [07/Jul/2026 12:10:26] "GET /static/images/background.jpg HTTP/1.1" 304 -
192.168.0.100 - - [07/Jul/2026 12:10:26] "GET /static/images/logo.png HTTP/1.1" 304 -
192.168.0.100 - - [07/Jul/2026 12:10:26] "GET /static/js/script.js HTTP/1.1" 304 -
192.168.0.100 - - [07/Jul/2026 12:10:27] "GET /favicon.ico HTTP/1.1" 404 -
```

Activity 5.3: Testing Model Accuracy

The trained Machine Learning model was evaluated using the testing dataset. Multiple classification algorithms were compared, and the model with the highest prediction accuracy was selected.

The selected model achieved excellent prediction performance and was saved for deployment within the Flask application.

This testing confirmed that the model provides reliable crop recommendations for agricultural data.

Model Comparison	
Algorithm	Accuracy
Decision Tree	0.9863636363636363
K-Nearest Neighbors (KNN)	0.9704545454545455
Logistic Regression	0.9522727272727273
Random Forest	0.9931818181818182

Activity 5.4: Testing Application Execution

The Flask application was executed locally to verify successful deployment.

The application was tested by running:

```
python app.py
```

The Flask development server started successfully without errors, and the web application became accessible through the local browser.

This confirmed that the frontend, backend, Machine Learning model, and prediction system were working together correctly.

Exploring the Website's Web Pages

Home Page

Description:

The Home page serves as the landing page of the OptiCrop application. It introduces the Smart Agricultural Production Optimization Engine and provides an overview of the project's purpose. The page includes a responsive navigation bar, project title, brief description, and call-to-action buttons that guide users to the crop prediction feature. The interface is designed using HTML, CSS, and JavaScript to provide a clean, modern, and user-friendly experience.



OptiCrop

Smart Agricultural Production Optimization Engine

OptiCrop is an AI-powered Smart Agricultural Production Optimization Engine that helps farmers identify the most suitable crop based on soil nutrients and environmental conditions using Machine Learning.

[Find Your Crop](#)

About OptiCrop

OptiCrop is a Machine Learning-based crop recommendation system that analyzes soil nutrients and environmental conditions to recommend the most suitable crop for cultivation.

The system considers Nitrogen (N), Phosphorus (P), Potassium (K), Temperature, Humidity, pH, and Rainfall to provide intelligent crop recommendations.

Our goal is to improve agricultural productivity, reduce crop failure, and encourage sustainable farming practices through Artificial Intelligence.

About Page

Description:

The About page provides detailed information about the OptiCrop project, including its objectives, features, technology stack, and the Machine Learning approach used for crop recommendation. It explains how the application predicts suitable crops based on soil nutrients and environmental conditions while highlighting the benefits of using Artificial Intelligence in agriculture.



About OptiCrop

OptiCrop is an AI-powered Smart Agricultural Production Optimization Engine that helps farmers identify the most suitable crop based on soil nutrients and environmental conditions using Machine Learning.



Project Objective

The objective of OptiCrop is to support farmers by recommending the most suitable crop based on Nitrogen (N), Phosphorus (P), Potassium (K), temperature, humidity, pH, and rainfall. The system aims to improve agricultural productivity, optimize resource utilization, and promote sustainable farming practices.



Key Features

- AI-based Crop Recommendation
- Analysis of Soil Nutrients (N, P, K)
- Weather and Environmental Analysis
- Crop Information with Growing Tips
- Simple, Fast and User-Friendly Interface



Technologies Used

- Python
- Flask
- Scikit-learn
- Pandas
- NumPy
- HTML5
- CSS3
- Bootstrap 5
- JavaScript



Team Members

- Mohammed Sadia Rahmatunnisa (Team Lead)
- Akhil Varma
- Done Divya
- Ullingala Swathi
- Yamini Karumuri



Our Mission

To empower farmers with intelligent crop recommendations using Artificial Intelligence and Machine Learning, helping them make informed decisions for higher productivity and sustainable agriculture.

[Home](#)[Find Your Crop](#)

Crop Prediction Page

Description:

The Crop Prediction page allows users to enter agricultural parameters such as Nitrogen, Phosphorus, Potassium, Temperature, Humidity, pH, and Rainfall. After submitting the input values, the application processes the data using the trained Machine Learning model and predicts the most suitable crop for cultivation. The page provides an easy-to-use interface with input validation to ensure accurate predictions.

Enter Soil Details

Nitrogen (N)

Phosphorus (P)

Potassium (K)

Temperature (°C)

Humidity (%)

pH Value

Rainfall (mm)

Predict Crop

Fill Sample Data

Clear Form

Prediction Result Page

Description:

The Prediction Result page displays the recommended crop generated by the Machine Learning model. In addition to the crop name, the application presents prediction confidence, crop image, suitable season, soil type, water requirement, fertilizer recommendation, harvest duration, and useful farming tips. This information helps farmers make informed decisions regarding crop cultivation and agricultural planning.

Prediction Result



Rice

The recommended crop based on the provided soil and environmental conditions.

Model Confidence

99.0%

Season	Kharif
Suitable Soil	Clay Loam
Temperature	20°C - 35°C
Humidity	80% - 90%
Rainfall	150 - 250 mm
Water Requirement	High
Harvest Time	120 - 150 Days
Fertilizer	Nitrogen-rich fertilizer
Growing Tips	Maintain standing water during early growth and use certified seeds.
Recommendation	Based on the provided soil nutrients and environmental conditions, this crop is the most suitable recommendation generated by the OptiCrop prediction model.

Conclusion

The OptiCrop – Smart Agricultural Production Optimization Engine was successfully developed using Machine Learning and the Flask framework to provide intelligent crop recommendations based on soil nutrients and environmental conditions. The application integrates data preprocessing, model training, backend development, and a responsive web interface to deliver accurate crop predictions in a simple and user-friendly manner.

The project demonstrates the practical application of Artificial Intelligence in agriculture by assisting farmers in selecting suitable crops, improving productivity, and supporting data-driven farming decisions. Through proper testing and implementation, the application achieved reliable prediction performance and successfully met the project objectives. Future enhancements may include weather forecasting integration, fertilizer recommendations, disease prediction, multilingual support, and cloud deployment to make the system more accessible and beneficial to a wider farming community.